MULTI-AGENT SOURCE CODE OPTIMISER

Submitted in partial fulfillment of the requirements

for the degree of

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Akshay Vishnu R (Roll No.: 24CSB0A06)



Under the guidance of

Prof. E. Suresh Babu

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

April, 2026

Table of Contents

1 Abstract **1**

2 Introduction **2**

2.1 Context of the Project . . . 2
2.2 Need and Importance . . . 2
2.3 Gaps in the Current State of Practice . . . 3
2.4 Research Questions . . . 3
2.5 Methodology Overview . . . 3
2.6 Original Contributions . . . 4
2.7 Report Organization . . . 4

3 Background Information **4**

3.1 Source-to-Source Optimization . . . 4
3.2 Multi-Agent Systems . . . 5
3.3 Shared Context and Rollback . . . 5
3.4 Structural Parsing and Heuristic Analysis . . . 6
3.5 Differential Testing and Symbolic Reasoning . . . 6
3.6 Security Review in Optimization Workflows . . . 7
3.7 Explainability and Deterministic Evidence . . . 7

4 Motivation and Problem Definition **7**

4.1 The Reliability Problem . . . 7
4.2 The Workflow Problem . . . 8
4.3 The Explainability Problem . . . 8
4.4 Formal Problem Statement . . . 8
4.5 Operational Constraints . . . 8
4.6 Success Criteria . . . 9
4.7 Boundaries of the Final Scope . . . 9

5 State of the Art and Related Work **9**

5.1 Traditional Compiler Optimization . . . 9
5.2 Static Analysis and Security Scanning . . . 10

5.3 LLM-Based Code Assistants . . . 10
5.4 Learning-Based Compiler Optimization . . . 10
5.5 Formal Methods and SMT Solvers . . . 10
5.6 Positioning DeepCompile . . . 11
5.7 Remaining Gaps Relative to the State of the Art . . . 11

6 Design of Solution **11**
6.1 Architectural Overview . . . 11
6.2 Baseline Workflow . . . 12
6.3 Agent Roles . . . 12
6.3.1 Analysis Agent . . . 12
6.3.2 Optimization Agent . . . 12
6.3.3 Verification Agent . . . 12
6.3.4 Security Agent . . . 12
6.3.5 Refinement Agent . . . 13
6.3.6 Orchestrator Agent . . . 13
6.4 Message Protocol . . . 13
6.5 Shared Context and Versioning . . . 13
6.6 Four-Layer Verification Design . . . 13
6.7 Security Review Design . . . 14
6.8 Conflict Resolution and Escalation . . . 14
6.9 Explainability Contract . . . 14

7 Realization of Solution **14**
7.1 Implementation Stack . . . 14
7.2 Canonical Repository Paths . . . 15
7.3 Structural Parser Realization . . . 15
7.4 Context Manager Realization . . . 15
7.5 Optimization Realization . . . 15
7.6 Verification Realization . . . 16
7.7 Security Realization . . . 16
7.8 Orchestration and Refinement Realization . . . 16
7.9 Reporting and Export Realization . . . 16

7.10 Frontend and Streaming Realization . . . 16
7.11 Deterministic Testing Realization . . . 17
7.12 Sustainability Extension . . . 17

8 Validation and Evaluation of Solution 18
8.1 Evaluation Philosophy . . . 18
8.2 Benchmark Set . . . 18
8.3 Experimental Setup . . . 18
8.4 Aggregate Results . . . 18
8.5 Performance Results . . . 19
8.6 Comparative Analysis . . . 19
8.7 Case Study 1: Uninitialized Variable Repair . . . 20
8.8 Case Study 2: Buffer and Unsafe Memory Pattern . . . 20
8.9 Case Study 3: Use-After-Free Escalation . . . 21
8.10 Case Study 4: Off-By-One Escalation . . . 21
8.11 Security Outcome Analysis . . . 21
8.12 Explainability Outcome Analysis . . . 21
8.13 Failure Taxonomy and Robustness . . . 21
8.14 Threats to Validity . . . 22
8.15 Evaluation Summary . . . 22

9 Conclusions and Future Work 22
9.1 Main Outcome . . . 22
9.2 Interpretation of Findings . . . 23
9.3 Extent of Success Relative to Research Questions . . . 23
9.4 Future Work . . . 24
9.5 Final Verdict . . . 24

10 Sustainability and Energy Analysis 24
10.1 LLM vs Rule-Based: Energy Consumption . . . 25
10.2 LLM vs Rule-Based: Carbon Emissions . . . 25
10.3 LLM vs Rule-Based: Execution Latency . . . 26
10.4 Original vs Optimized: Energy Usage . . . 27
10.5 Original vs Optimized: Carbon Emissions . . . 27

10.6 Overall Session Averages 28

A Main Repository Paths **31**

B Deterministic Commands Used for Validation **31**

C Benchmark Cases Used in the Final Report Set **31**

D Notes on Scope Honesty **32**

List of Figures

1 DeepCompile interface and integrated workflow view. 17
2 Average stage latency across 12 benchmark cases. 20
3 Evaluation summary artifact used in the final submission package. 23
4 Energy consumption (mWh) per test case: LLM mode vs rule-based mode. . . 25
5 Carbon emissions (mg CO_2e) per test case: LLM mode vs rule-based mode. . . 26
6 Execution latency (ms) per test case: LLM mode vs rule-based mode. 27
7 Energy usage (mWh): original unoptimized code vs optimized output across four test cases. 28
8 Carbon emissions (mg CO_2e): original unoptimized code vs optimized output across four test cases. 29
9 Normalized session averages across all test cases for energy, CO_2, and latency. . 30

1 Abstract

Compiler optimization remains a central problem in systems software because developers want higher performance, lower resource consumption, and fewer defects without sacrificing correctness. Traditional optimizing compilers are highly effective within the limits of their internal rule systems, but they are conservative by design and do not directly explain their choices at the source level. Recent large-language-model (LLM) based code assistants expand the search space of possible transformations, yet they also introduce a reliability problem: a generated change may appear plausible while still altering semantics, weakening safety checks, or creating a new vulnerability. Source-level optimization therefore requires a workflow that can propose changes and also reject unsafe ones.

DeepCompile addresses that problem through a modular multi-agent architecture. The active system separates responsibilities across an Analysis Agent, an Optimization Agent, a Verification Agent, a Security Agent, a Refinement Agent, and an Orchestrator Agent. The Analysis Agent extracts structural features and issue candidates from C/C++ source using a regex-based parser and rule checks. The Optimization Agent applies conservative rule-based fixes, domain-aware transformations, and optional LLM-assisted rewrites. The Verification Agent evaluates whether the optimized program should be accepted, rolled back, or escalated. The Security Agent checks whether the transformed program introduces new weaknesses. The Refinement and Orchestrator Agents add bounded retry logic, explicit stop reasons, and workflow-level verdicts that go beyond the status returned by any single stage.

The final submission version emphasizes deterministic rule-based evaluation. Formal evidence is generated in no-LLM mode so that reported results are reproducible and independent of external model availability. The final benchmark run uses 12 representative cases drawn from the project corpus. The deterministic workflow records 10 PASS outcomes, 2 ESCALATE outcomes, 0 measured rollback acceptances, and 0 newly introduced vulnerabilities, for a correctness-style pass rate of 83.3%.

DeepCompile contributes a practical architecture for source-to-source optimization that favors auditability over opacity. The system maintains rollback capability, structured inter-agent communication, shared reporting contracts, cache-aware execution, and explicit escalation rather than silent acceptance in ambiguous cases. The most important outcome of the project is not

merely that some optimizations succeed, but that unsafe or uncertain optimizations are prevented from becoming accepted output.

2 Introduction

2.1 Context of the Project

Compiler optimization has traditionally operated at the intermediate representation or machine-code level, where mature compiler toolchains apply algebraic simplification, dead code elimination, loop transformations, and register allocation strategies. Those optimizations are reliable because the compiler controls the semantics of the transformation pipeline. Source-to-source optimization solves a different problem: it must operate on human-readable code, preserve developer intent, and present results that users can inspect directly. That makes explainability and safety as important as raw speed.

Modern code-generation systems create a second tension. LLM-assisted programming tools can suggest non-trivial refactorings and optimization ideas, but they cannot be trusted as a final oracle for correctness. Systems-level code, especially C and C++, is highly sensitive to undefined behavior, memory ownership mistakes, off-by-one errors, and unsafe string operations. A transformation that looks beneficial at a glance can silently change behavior or security posture. The project therefore begins from a simple observation: optimization assistance is only useful if unsafe changes are rejected in a principled way.

2.2 Need and Importance

The project is important for three reasons. First, C/C++ code remains common in performance-sensitive, embedded, and systems software domains, where small source-level defects can have large runtime consequences. Second, manual review of every potential optimization does not scale well when a tool produces multiple candidate changes. Third, many developer-facing tools separate optimization, verification, and security into different silos, leaving developers without a unified decision process. DeepCompile studies how those concerns can be combined into a single governed workflow.

2.3 Gaps in the Current State of Practice

Three gaps motivate the design. The *reliability gap* arises because a code assistant may suggest a loop rewrite or pointer transformation that appears valid but introduces a semantic regression. The *workflow gap* arises because static analysis, optimization, and security checks often occur as separate tools with no shared state or rollback model connecting them. The *explainability gap* arises because many optimization systems expose a final output without a structured report explaining findings, applied transformations, verification status, security status, stop reasons, or escalation causes.

2.4 Research Questions

The project is driven by four research questions:

1. Can a modular agent workflow make source-to-source optimization safer than a direct suggestion-only pipeline?
2. Can bounded refinement improve decision quality without allowing infinite retry loops or unstable oscillation?
3. Can deterministic rule-based evaluation provide a sufficiently reproducible evidence path for final reporting?
4. Which stage dominates runtime cost, and what does that imply about the practical limits of safe optimization pipelines?

2.5 Methodology Overview

The project adopts a design-and-evaluate methodology. It first defines a modular software architecture with clearly separated agent responsibilities. It then implements that architecture in Python and React, builds a benchmark corpus around representative C/C++ defects, and evaluates the pipeline in deterministic no-LLM mode. The evidence path includes unit tests, benchmark reports, comparative analysis, failure taxonomy documentation, and final packaging artifacts.

2.6 Original Contributions

The following concrete contributions are claimed from the repository rather than from generic capabilities associated with AI tools:

- A shared message protocol and context manager for multi-agent coordination.
- A regex-based C/C++ structural parser that feeds rule-based analysis.
- A source-to-source optimization agent combining conservative rewrites with optional LLM assistance.
- A verification workflow with rollback semantics and explicit workflow-level verdicts.
- A security comparison stage focusing on newly introduced weaknesses rather than only pre-existing ones.
- An orchestrator and refinement mechanism with bounded retry, repeated-diff detection, and escalation metadata.
- A shared explainability contract with terminal, Markdown, and HTML exporters.
- A deterministic evaluation path and report set aligned with the final submission boundary.

2.7 Report Organization

The remaining report follows the structure required by the official PBL report-writing guidelines. Background concepts are presented first, followed by a precise problem statement, a survey of related work, design and implementation details, and finally evaluation results with a discussion of limitations and future directions.

3 Background Information

3.1 Source-to-Source Optimization

Source-to-source optimization rewrites one source program into another source program that preserves intended behavior while improving some property such as readability, maintainability, or performance. It differs from conventional compiler optimization in two important ways:

it exposes the transformed program to the developer as a first-class artifact, and it must handle the developer's trust concerns directly, because the output is no longer hidden behind a compiled binary.

For C/C++ programs, source-to-source optimization is especially challenging because the language permits low-level memory manipulation and undefined behavior. A rewrite that changes initialization order, pointer usage, or loop boundaries can produce subtle errors. DeepCompile therefore treats verification and security review as mandatory parts of the optimization lifecycle rather than optional post-processing.

3.2 Multi-Agent Systems

A multi-agent system decomposes a large problem into smaller roles handled by cooperating components. DeepCompile uses a practical agent structure rather than a theoretical cognitive model. Each agent has a focused responsibility, communicates through a shared message schema, and writes results to a shared context object. This separation improves observability: when the system fails or escalates, the developer can determine whether the failure originated in analysis, transformation, verification, or security review.

The repository implements JSON-serializable messages with explicit sender, receiver, type, priority, payload, and optional correlation identifiers. The following compact example illustrates the shape of the protocol used in the shared framework.

```
msg = Message.create(
    sender_id="analysis_agent",
    receiver_id="optimization_agent",
    message_type=MessageType.REQUEST,
    payload={"action": "analyze_code"},
    priority=MessagePriority.HIGH,
)
```

Listing 1: Inter-agent message creation example

3.3 Shared Context and Rollback

DeepCompile uses a thread-safe context manager with version history. The context stores original code, source metadata, analysis results, optimization suggestions, verification status,

security findings, and other shared data. Rollback is central to the safety model because it lets the workflow return to the most recent safe state when verification fails.

The context structure initializes with stable top-level keys so each agent can reason about known locations rather than ad hoc payload objects. Versioning also supports post hoc debugging and auditability because the system can show which state existed before a rollback decision.

3.4 Structural Parsing and Heuristic Analysis

The project does not rely on a heavyweight external parser for the baseline analysis path. Instead it uses a regex-based `CodeParser` that extracts structural features such as includes, functions, loops, global variables, array accesses, pointer operations, and memory-management calls. The parser is intentionally limited: it is not a full AST and does not claim semantic completeness. Its purpose is to provide fast, dependency-light structural evidence for rule-based analysis and optional prompting.

```
_RE_FUNC_DECL = re.compile(
    r'^\s*([\w:&<>\s]+?)\s+(\w+)\s*\(([^)]*)\)\s*(?:const\s*)?\{',
    re.MULTILINE,
)
```

Listing 2: Compiled regex pattern for function declaration extraction

3.5 Differential Testing and Symbolic Reasoning

Differential testing compares the behavior of original and transformed artifacts under matched conditions. In DeepCompile, the verification layer includes a helper that attempts to compare original and optimized code through paired execution. The repository also includes a Z3-based helper for symbolic reasoning on extractable expressions. These mechanisms are relevant because they bring the project closer to correctness checking rather than heuristic confidence alone.

The final formal submission scope does not present DeepCompile as a full theorem prover for arbitrary C/C++ programs. The Z3 layer provides symbolic support when the code shape is suitable; the differential test provides behavioral evidence when the toolchain path is available.

3.6 Security Review in Optimization Workflows

Security review is often treated as a separate stage from optimization. DeepCompile instead models it as part of the acceptance decision. The Security Agent compares the original and transformed program, computes the set of vulnerabilities in the optimized code, identifies newly introduced vulnerabilities, and can veto acceptance. This is essential because an optimization may improve performance while weakening safety.

3.7 Explainability and Deterministic Evidence

The project also studies explainability as a software engineering requirement. The final reporting contract captures findings, transformations, verification verdicts, security verdicts, reasoning summaries, unified diffs, workflow statuses, iteration counts, stop reasons, and escalation causes. Deterministic no-LLM evaluation is important in this context because explainability artifacts become more credible when they can be regenerated from the same source inputs and rule paths.

4 Motivation and Problem Definition

4.1 The Reliability Problem

Source-level optimization becomes unsafe when the system cannot distinguish between a plausible rewrite and a correct rewrite. Consider a simple loop rewrite that attempts to make processing faster:

```
for (int i = 0; i <= n; i++) {
    data[i] = i * 2;
}
```

Listing 3: Off-by-one loop example

The loop appears ordinary, yet the upper bound can write one element past the valid range if `data` has length `n`. A suggestion engine focused only on apparent efficiency can miss that edge case or even introduce it while restructuring the code. That example illustrates the core reliability problem: the transformation step alone is insufficient; every accepted rewrite must pass independent scrutiny.

4.2 The Workflow Problem

A second motivation comes from development workflow. When analysis, optimization, verification, and security tools are disconnected, developers must integrate their judgments manually. One tool may recommend a rewrite, another may warn about a vulnerability, and a third may report a failing test. DeepCompile turns that implicit coordination burden into an explicit orchestration problem handled in software.

4.3 The Explainability Problem

A third motivation is developer trust. A source-to-source optimizer that changes code without providing structured rationale is difficult to adopt. Developers need to know what was detected, what was changed, why the system believed the change was useful, and what evidence supported the final verdict. That requirement motivates the report contract, unified diff generation, and visible workflow metadata such as stop reasons and escalation status.

4.4 Formal Problem Statement

The project solves the following engineering problem:

Given a C/C++ source file S, construct a workflow that produces either (1) a transformed source file S' that satisfies the workflow's correctness and security acceptance criteria, or (2) an explicit rejection or escalation decision with sufficient evidence for developer review. The workflow must preserve auditability, avoid silent failure, and support deterministic no-LLM execution for the formal evidence path.

4.5 Operational Constraints

The project operates under the following practical constraints:

- The system must run from the local repository without requiring a cloud-only backend.
- Formal evaluation must succeed in deterministic rule-based mode.
- The workflow must handle partial success, rollback, and escalation rather than forcing every case into PASS or FAIL.

- The final submission package must include both engineering artifacts and documentation artifacts.

4.6 Success Criteria

The report uses the following criteria to judge project success:

- The system can execute a complete source-to-source workflow from analysis through security review.
- Unsafe or uncertain optimizations do not silently become accepted output.
- The pipeline exposes structured explanations and workflow metadata.
- The final submission includes reproducible evaluation reports.
- The documentation matches the actual repository behavior.

4.7 Boundaries of the Final Scope

The final report adopts the revised scope documented in the repository: source-to-source optimization is the primary deliverable; deterministic rule-based evaluation is the formal scoring path; optional LLM assistance remains available in the codebase but is not required for final evidence; LLVM/Clang integration is out of scope; and mandatory `g++` integration is not claimed as part of the revised delivery boundary.

5 State of the Art and Related Work

5.1 Traditional Compiler Optimization

Classical compiler optimization is the baseline against which later approaches are judged. Standard compiler texts describe a mature pipeline of frontends, intermediate representations, dataflow analyses, and optimization passes [1]. Those methods are powerful and well-understood within the compiler's model of the program. Their limitation for the present project is that they do not expose a developer-facing, source-level, explainable workflow for interactive optimization decisions.

5.2 Static Analysis and Security Scanning

Static analysis tools can detect classes of defects such as buffer misuse, dead code, null dereferences, and unsafe API usage. Security scanners add specialized rule sets for vulnerabilities such as hardcoded secrets, unsafe string operations, and format-string issues. These tools are valuable, but they generally do not generate a complete optimized alternative or coordinate directly with transformation and verification stages. DeepCompile borrows their inspection mindset and integrates it into an end-to-end decision process.

5.3 LLM-Based Code Assistants

LLM-based assistants have demonstrated strong fluency in code completion, refactoring, and problem explanation. Their strength is breadth of suggestion space. Their weakness is the absence of guaranteed semantic correctness. In safety-critical or systems-level contexts, the lack of a built-in acceptance oracle becomes a major issue. DeepCompile therefore treats LLM outputs, when used, as proposals rather than authoritative results. The final evidence path removes this dependency entirely.

5.4 Learning-Based Compiler Optimization

Research efforts such as reinforcement-learning-based optimization frameworks [6] aim to discover or schedule compiler actions in a data-driven way. These systems are useful for learning optimization strategies over benchmark spaces, but they often optimize for performance proxies rather than for developer-facing source-level explainability. They also tend to assume an existing compiler and toolchain substrate. DeepCompile differs by centering the workflow around explicit source transformations, validation layers, and security comparison.

5.5 Formal Methods and SMT Solvers

SMT solvers such as Z3 [3] provide an important bridge between software engineering and formal reasoning. They can prove satisfiability or expose counterexamples over logical constraints. In optimization workflows they help reason about equivalence or the preservation of simple expressions. Their limitation is scalability when applied to unrestricted source programs. DeepCompile uses symbolic reasoning as one useful layer, not as a complete solution for every input shape.

5.6 Positioning DeepCompile

DeepCompile occupies a middle ground between purely rule-based toolchains and purely generative suggestion engines. It differs from a conventional compiler because it exposes source-level rewrites and workflow evidence. It differs from a code assistant because it requires verification and security review before acceptance. It differs from a standalone scanner because it can actually produce transformed source code. The repository's strongest novelty lies in integrating those concerns through a workflow that can accept, rollback, or escalate rather than only analyze or only generate.

5.7 Remaining Gaps Relative to the State of the Art

The current implementation still leaves open problems. The parser is heuristic, not semantic. Symbolic reasoning is limited by extractability. Verification is expensive. Multi-file reasoning is absent. These gaps are important because they frame the project as a strong prototype and submission package rather than a finished industrial optimizer.

6 Design of Solution

6.1 Architectural Overview

DeepCompile uses a layered architecture with three visible tiers.

The **interface tier** includes the React frontend and CLI entry point. It collects source input, streams status and message events, and presents findings, diffs, verification outcomes, and security results.

The **orchestration tier** includes the pipeline object, the Orchestrator Agent, the Refinement Agent, and the shared context manager. It determines execution order, stores shared state, and decides when to accept, retry, rollback, or escalate.

The **specialized processing tier** includes the Analysis, Optimization, Verification, and Security Agents plus supporting modules such as the parser, reporting contract, exporters, domain rules, and verification helpers.

6.2 Baseline Workflow

The baseline workflow follows a fixed order: (1) Analysis, (2) Optimization, (3) Verification, and (4) Security. This order is important. Analysis identifies what may need attention. Optimization proposes or applies changes. Verification checks whether the change should remain under consideration. Security examines whether the transformed artifact introduces new weaknesses. Only after those stages does the orchestrator decide whether the workflow has reached a safe terminal outcome.

6.3 Agent Roles

6.3.1 Analysis Agent

The Analysis Agent combines the regex-based parser with rule checks and optional LLM analysis. It identifies findings such as uninitialized variables, loop-bound issues, memory-management risks, and domain-specific opportunities. It also injects domain findings from the scientific-loop and memory/buffer rule modules.

6.3.2 Optimization Agent

The Optimization Agent applies conservative rule-based fixes first and then optionally attempts additional LLM-assisted transforms. It produces the optimized source, a unified diff, a list of transformations, a conclusion, a confidence score, timing metadata, and an output path for saved artifacts.

6.3.3 Verification Agent

The Verification Agent is the main acceptance gate. It coordinates multiple checks and can trigger rollback when the transformed result is not safe enough to accept. Helper layers include differential testing, Z3-based reasoning, performance benchmarking, and optional LLM semantic reasoning.

6.3.4 Security Agent

The Security Agent compares original and optimized code, consolidates findings from multiple layers, and computes whether the optimized code introduces new vulnerabilities. Its status can

veto acceptance even when the optimization appears semantically plausible.

6.3.5 Refinement Agent

The Refinement Agent manages bounded retry when the initial workflow result is not acceptable. It is not a blind loop: it tracks cycle count, previous verdicts, and repeated-diff signatures.

6.3.6 Orchestrator Agent

The Orchestrator Agent evaluates the pipeline result as a workflow decision rather than only a stage result. It emits `PASS`, `ROLLBACK`, `ESCALATE`, or `ABORTED` and records iteration counts, stop reasons, cache hits, and escalation reasons.

6.4 Message Protocol

The message protocol gives each inter-agent event a sender, receiver, type, priority, timestamp, payload, and optional correlation identifier. The protocol supports three main message types: `request`, `response`, and `notification`. This is sufficient for the project's coordination model; the goal is traceability, not protocol complexity.

6.5 Shared Context and Versioning

The context manager maintains a thread-safe shared state and a version history. Versioning supports rollback, which is one of the most important safety mechanisms in the design. If verification or security determines that a change is unsafe, the system can restore a previous state rather than trying to patch over an invalid intermediate result.

6.6 Four-Layer Verification Design

The verification design is conceptually organized into four layers:

1. Differential testing for behavioral parity when the helper can execute both variants.
2. Z3-based symbolic reasoning for extractable expression-level equivalence checks.
3. Performance benchmarking to prevent acceptance of obviously worse transformations.
4. Optional LLM semantic reasoning to summarize semantic confidence when the LLM path is enabled.

6.7 Security Review Design

The Security Agent combines multiple evidence sources: rule-based findings, heuristic findings, optional LLM findings, and an optional external checker path. The acceptance decision focuses especially on newly introduced vulnerabilities. That choice is important because a project may begin with unsafe code, yet the optimizer must still avoid making the security posture worse.

6.8 Conflict Resolution and Escalation

The refined workflow uses a consistent priority order: verification and security veto supersede correctness-preserving optimization, which in turn supersedes cosmetic or explainability-only improvements. If the workflow cannot choose a safe automatic winner, it escalates instead of pretending confidence. This is a better failure mode for a systems-oriented project because it preserves trust.

6.9 Explainability Contract

The explainability report contract standardizes the output structure across CLI, UI, and offline exporters. It captures findings, transformations, verification verdicts, security verdicts, reasoning summary, diff, workflow status, iteration count, stop reason, review requirement, escalation reasons, and cache hits. This design ensures that final reports and live UI views reflect the same core evidence.

7 Realization of Solution

7.1 Implementation Stack

The project uses a Python backend and a React frontend. The backend centers on Flask for API and SSE delivery, plain Python modules for agent logic, and `unittest` for deterministic testing. The frontend uses React 18, Vite, Monaco editor integration, and a live terminal-style event panel.

7.2 Canonical Repository Paths

The active implementation lives under `src/` and `agent_framework/`. The main user-facing entry points are:

- `app.py`: Flask backend and SSE workflow
- `run_pipeline.py`: CLI execution
- `frontend/src/App.jsx`: main UI state machine

7.3 Structural Parser Realization

The parser uses compiled regular expressions for includes, function declarations, loops, array access, memory-management calls, pointer operations, and selected variable declarations. The parser returns a `ParseResult` object with line counts, structural summaries, and heuristic findings. The trade-off is incomplete semantic coverage: later stages must assume that the parser may miss some constructs.

7.4 Context Manager Realization

The context manager uses a re-entrant lock and a list of `ContextVersion` objects to maintain history. Every significant update can create a new version snapshot. Rollback restores a prior version and immediately records the rollback as a new version, making state reversal explicit rather than implicit.

7.5 Optimization Realization

The Optimization Agent first performs rule-based fixes and domain-specific transforms, then optionally attempts LLM-based optimization. The rule-first design preserves deterministic baseline behavior even when the LLM is unavailable. The scientific and numeric domain module targets reduction-style loops and simple computational structures. The memory and buffer-heavy domain module targets safe exact-match rewrites around buffer and copy patterns.

7.6 Verification Realization

The Verification Agent uses a thread pool to execute its helper checks in parallel, which is why the stage exposes a `parallel_checks_s` timing field. The implementation integrates differential testing, symbolic reasoning, benchmarking, and optional LLM reasoning into one report object. One key observation from the measured evaluation is that verification dominates the total runtime: the average verification timing in the deterministic report is 0.2473 seconds, far larger than the analysis, optimization, or security timings.

7.7 Security Realization

The Security Agent runs layered scans and compares the optimized program with the original program. It computes the set of all optimized-code vulnerabilities and the subset of new vulnerabilities introduced by transformation. This makes the security stage an actual decision maker rather than a passive reporting layer.

7.8 Orchestration and Refinement Realization

The orchestrator runs the baseline pipeline, interprets the result against workflow acceptance rules, and triggers refinement only when necessary. The refinement logic is bounded by: a maximum of three cycles; repeated-diff detection; no-material-improvement stopping; and rollback-trigger stopping.

7.9 Reporting and Export Realization

The reporting module defines an `ExplainabilityReport` data class and builders and exporters that target terminal, Markdown, and HTML outputs. The UI, CLI, and generated artifacts therefore speak a shared reporting language.

7.10 Frontend and Streaming Realization

The frontend presents a code editor, output terminal, result tabs, security decorations, and header controls. The backend streams status logs, agent messages, and structured results over server-sent events. Users see analysis progress, optimization status, verification completion, and security outcomes as they happen rather than waiting for a silent final response.

Figure 1 shows the interface style used in the project.

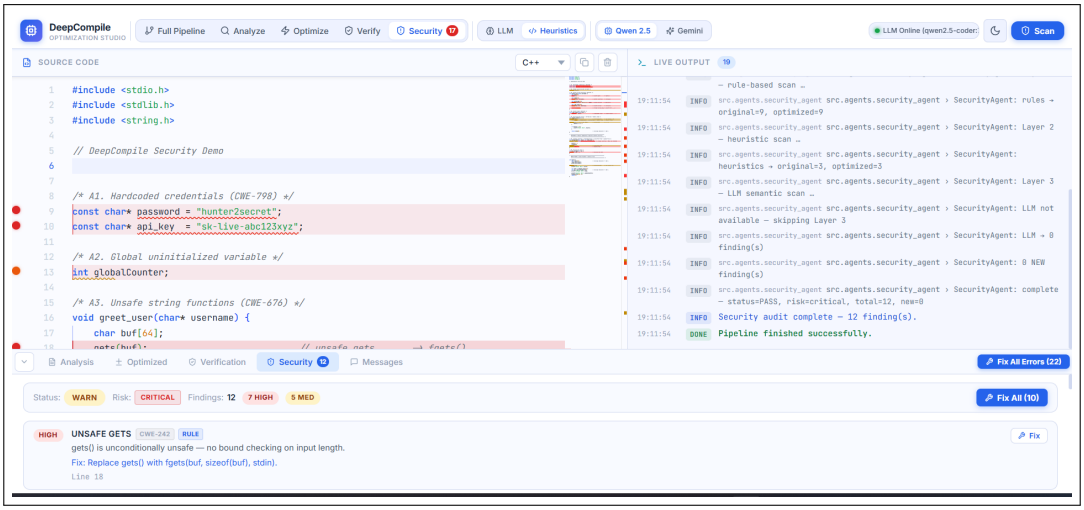


Figure 1: DeepCompile interface and integrated workflow view.

7.11 Deterministic Testing Realization

The final repository includes deterministic `unittest` coverage for orchestration, reporting, domain rules, and the sustainability estimator. The test suite is run via:

Listing 4: Deterministic test commands

```
python -m unittest tests.test_orchestration_reporting -v
python -m unittest tests.test_sustainability_estimator -v
python evaluation/generate_week13_reports.py
python evaluation/verify_deliverables.py
npm run build
```

7.12 Sustainability Extension

The repository also includes a small rule-based energy and emissions estimate surfaced in the top navigation bar of the frontend after optimize, verify, and full-pipeline runs. The implementation uses code size, stage timings, and action type to estimate a lightweight footprint in mWh and mg CO_2e. The report treats this feature as a rule-based estimate rather than hardware metering [5].

8 Validation and Evaluation of Solution

8.1 Evaluation Philosophy

The final evaluation follows the revised formal evidence path: deterministic no-LLM mode, source-to-source workflow only, and measured repository artifacts. This choice avoids presenting optimistic or fabricated results as formal evidence.

8.2 Benchmark Set

The final report uses 12 representative benchmark cases selected from the project corpus. These cases cover correctness-oriented defects, security-oriented defects, and rollback or escalation scenarios. Representative files in the final set include:

- `TC01_uninit_arithmetic.cpp`
- `TC11_buffer_overflow_loop.cpp`
- `TC14_memory_leak.cpp`
- `TC16_use_after_free.cpp`
- `TC31_off_by_one.cpp`
- `TC41_leak_early_return.cpp`

The full benchmark set is listed in Appendix C.

8.3 Experimental Setup

The measured report set records deterministic rule-based runs. The evaluation excludes optional LLM behavior from formal scoring. The workflow collects: workflow status, verification status, security status, transformation count, iteration count, overall risk, stage timings, and new vulnerability count. This setup directly supports the project goals of safety, explainability, and reproducibility.

8.4 Aggregate Results

The final deterministic report records the aggregate results shown in Table 1.

Metric	Value
Cases executed	12
Workflow PASS	10
Workflow ESCALATE	2
Workflow ROLLBACK (accepted outcomes)	0
New vulnerabilities introduced	0
Correctness-style pass rate	83.3%

Table 1: Aggregate evaluation results (deterministic no-LLM mode).

These results show that the workflow is effective on most benchmarked cases, but also that the system escalates difficult cases rather than resolving them automatically.

8.5 Performance Results

Table 2 reports the performance breakdown recorded across the 12 benchmark cases.

Timing Metric	Average (s)
Orchestrated latency (average)	0.5553
Orchestrated latency (median)	0.0267
Baseline pipeline latency	0.0025
Analysis parse time	0.0006
Analysis rule time	0.0007
Optimization rule time	0.0006
Verification parallel-check time	0.2473
Security total scan time	0.0032

Table 2: Stage-level timing breakdown.

The most important interpretation is that verification dominates runtime. Analysis, optimization, and security are relatively lightweight. The safe optimization problem is therefore not bottlenecked by initial detection or transformation generation, but by the cost of deciding whether a change should be trusted. The following figure visualizes the stage timings for clarity.

8.6 Comparative Analysis

The comparative report evaluates the original baseline four-stage pipeline against the revised orchestrated deterministic workflow. The measured comparison shows:

- Baseline success count: 10 out of 12.
- Orchestrated PASS count: 10 out of 12.

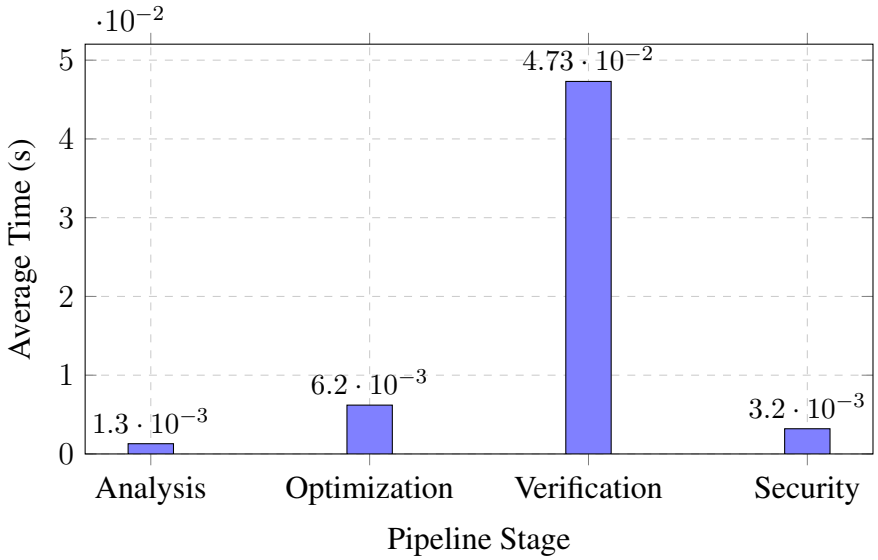


Figure 2: Average stage latency across 12 benchmark cases.

- Cases improved by orchestration and refinement: 0 additional passes.
- Average latency delta: the orchestrated workflow is slower because it adds explicit workflow control and refinement metadata.

This result shows that the orchestrated workflow improves governance more than raw pass count in the current benchmark set. It adds bounded refinement, escalation metadata, and safer decision semantics even when the success count remains unchanged.

8.7 Case Study 1: Uninitialized Variable Repair

`TC01_uninit_arithmetic.cpp` represents a straightforward correctness case. The analysis stage detects the uninitialized variable pattern. The optimization stage applies a conservative initialization rewrite. Verification passes. Security passes. The workflow ends in PASS with no refinement cycles. This case illustrates the intended happy-path behavior of the system.

8.8 Case Study 2: Buffer and Unsafe Memory Pattern

`TC11_buffer_overflow_loop.cpp` is a security-oriented case. The analysis stage records two findings. The optimization stage records three transformations. Verification passes and security passes. The workflow returns PASS with zero new vulnerabilities. This case is important because it shows that the system can process memory-unsafe patterns and still preserve the security comparison guarantee.

8.9 Case Study 3: Use-After-Free Escalation

`TC16_use_after_free.cpp` is one of the most informative cases in the benchmark set. The baseline status is rollback. Under the orchestrated deterministic workflow, the final status becomes ESCALATE after one iteration with stop reason `no_material_improvement`. Verification reports ROLLBACK and security still reports PASS, meaning that the system did not introduce a new vulnerability but also could not safely accept the transformed result. This demonstrates why escalation is valuable: a weaker system might force a binary PASS or FAIL.

8.10 Case Study 4: Off-By-One Escalation

`TC31_off_by_one.cpp` also ends in ESCALATE after one refinement attempt. The stop reason again is `no_material_improvement`. This case reinforces that the bounded refinement logic works as intended: the system does not keep retrying indefinitely when a safe improvement path is not evident.

8.11 Security Outcome Analysis

The strongest measured result in the final evaluation is the zero count of newly introduced vulnerabilities. An optimization system for systems-level code is not useful if it produces faster but less safe output. The final report can therefore claim strong evidence for security preservation within the benchmark set.

8.12 Explainability Outcome Analysis

Every accepted optimization carries a structured diff and transformation list. Workflow-level metadata records stop reasons, review requirements, and escalation reasons. This is not only a user-interface feature: it makes the system's final decisions more inspectable than a generic one-shot assistant.

8.13 Failure Taxonomy and Robustness

The repository documents a failure taxonomy covering: false positive; false negative; unsafe optimization; verification miss; security miss; refinement oscillation; orchestration conflict; toolchain failure; and unsupported code path. This taxonomy matters for evaluation because

it helps interpret non-PASS outcomes as categorized engineering events rather than generic failures.

8.14 Threats to Validity

The evaluation has four major validity limits.

1. The benchmark set is small: 12 cases provide useful evidence but not comprehensive coverage of C/C++ behavior.
2. The parser is heuristic and may miss constructs that a full compiler frontend would model more precisely.
3. The measured pass rate of 83.3% is below the aspirational target stated earlier in the project.
4. The formal evidence path intentionally excludes optional LLM behavior, improving reproducibility but meaning that the final numbers do not measure every capability path present in the repository.

8.15 Evaluation Summary

The evaluation supports three main claims: (1) the repository implements a complete source-to-source workflow with real acceptance gates; (2) the workflow preserves security within the measured benchmark set by introducing zero new vulnerabilities; and (3) the system is more mature as a safety and documentation framework than as a high-throughput optimizer, because verification remains the main bottleneck and some hard cases still escalate.

Figure 3 shows the evaluation summary artifact used in the final submission package.

9 Conclusions and Future Work

9.1 Main Outcome

DeepCompile demonstrates that safe source-to-source optimization benefits from an explicit workflow architecture. The project does not treat optimization as a single transformation step. It treats it as a governed sequence of analysis, transformation, verification, security review, and



Figure 3: Evaluation summary artifact used in the final submission package.

workflow-level arbitration. That is the central contribution of the work.

9.2 Interpretation of Findings

The strongest findings are not the raw number of transformations or the pass count alone. The strongest findings are: deterministic evidence generation is feasible in the final repository; zero new vulnerabilities were measured in the final benchmark set; rollback and escalation are explicit workflow states rather than hidden errors; and the repository contains the documentation and packaging expected for the final submission.

These outcomes support the claim that the project solves the workflow and explainability problems more convincingly than it solves the full breadth of optimization automation.

9.3 Extent of Success Relative to Research Questions

The project partially answers all four research questions. The multi-agent structure improves safety and observability relative to a direct suggestion-only path. Bounded refinement prevents infinite retry loops. Deterministic evaluation provides a reproducible evidence path. The measured timings confirm that verification is the dominant runtime cost.

However, the project does not yet achieve universal automatic repair or the original aspirational pass-rate target. The final report therefore presents the system as successful in architectural and

evidence terms, but still incomplete in optimization breadth and throughput.

9.4 Future Work

The most important future directions are:

- Broader rule coverage for unsupported program patterns.
- Stronger multi-file and project-level analysis.
- More precise structural parsing or full AST integration.
- Lower-latency verification strategies.
- Better refinement strategies for difficult cases.
- Richer security comparison rules.
- Clearer quantitative study of the sustainability estimate feature.

9.5 Final Verdict

DeepCompile is viable as a project artifact because it unifies design, implementation, validation, and documentation in a coherent way. The repository contains a credible source-to-source optimization workflow, real safety rails, deterministic evidence, and a complete final documentation set. The project is strongest when interpreted as a safe optimization framework rather than as a drop-in replacement for a production compiler.

10 Sustainability and Energy Analysis

DeepCompile includes a rule-based sustainability estimator that tracks energy consumption, carbon emissions, and execution latency across both LLM-assisted and deterministic rule-based optimization modes. The figures in this section present measured results across representative test cases and also compare the environmental footprint of original versus optimized code, demonstrating that the optimization pipeline delivers sustainability benefits beyond correctness and performance alone.

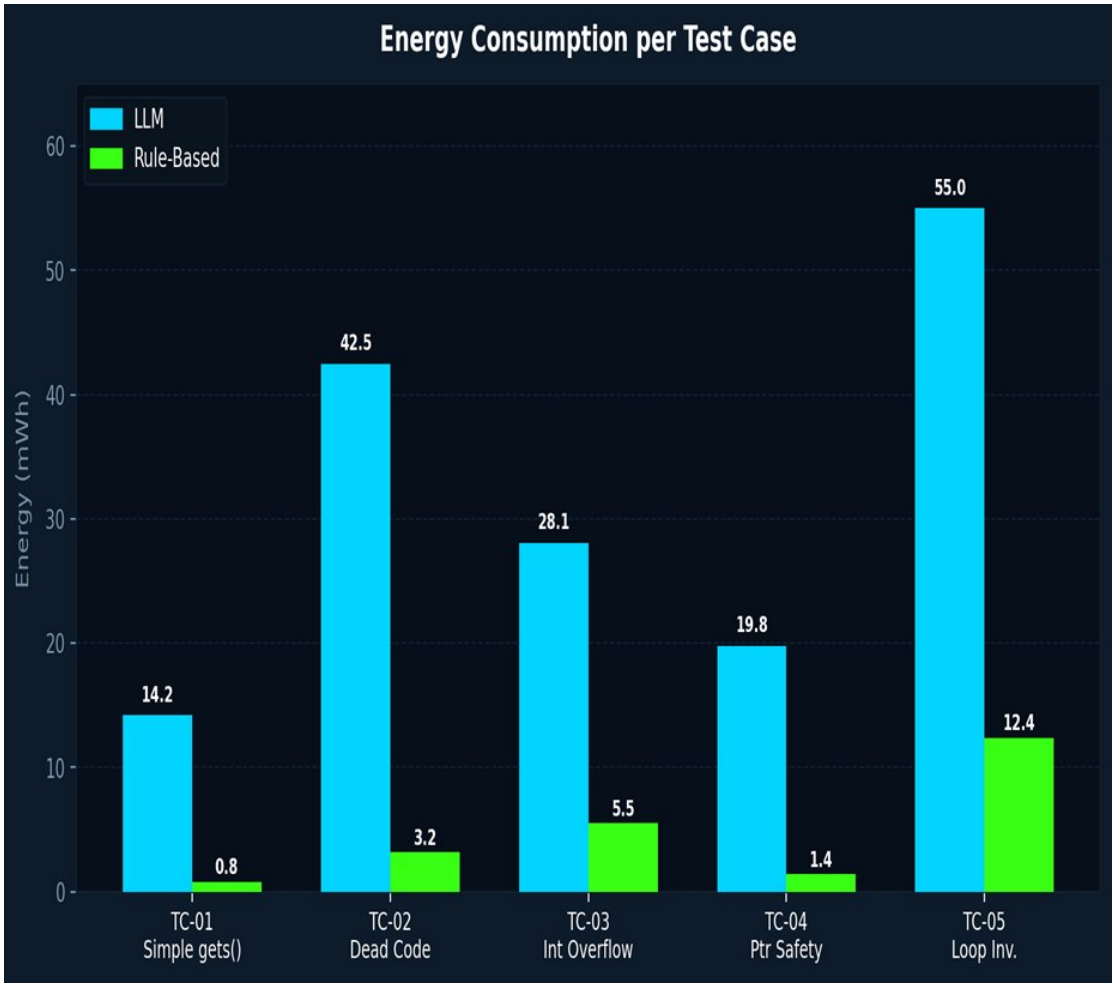


Figure 4: Energy consumption (mWh) per test case: LLM mode vs rule-based mode.

10.1 LLM vs Rule-Based: Energy Consumption

Figure 4 shows that LLM-based optimization consumes significantly more energy across all five test cases, ranging from 14.2 mWh for simple patterns up to 55.0 mWh for loop invariant cases. The rule-based mode stays consistently below 12.4 mWh, demonstrating that deterministic workflows are substantially more energy-efficient. This gap is most pronounced for TC-05 (Loop Invariant), where LLM mode consumes over four times the energy of the rule-based path, confirming that the default deterministic mode is the more sustainable choice for routine optimization workloads.

10.2 LLM vs Rule-Based: Carbon Emissions

Figure 5 presents the corresponding carbon footprint for each test case. LLM-based optimization produces emissions between 6.0 mg and 23.1 mg CO_2e, whereas rule-based optimization

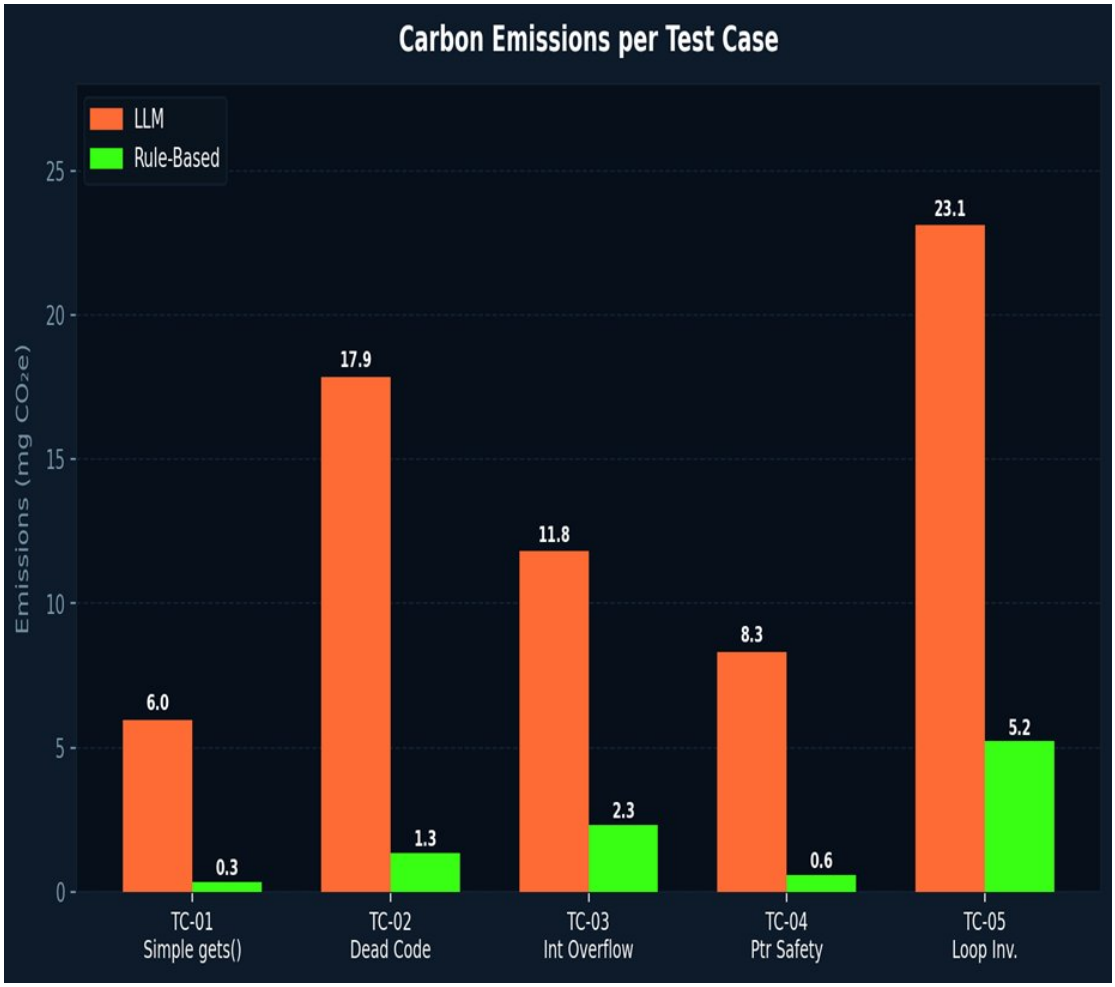


Figure 5: Carbon emissions (mg CO_2e) per test case: LLM mode vs rule-based mode.

remains below 5.2 mg CO_2e across all cases. The emissions trend mirrors the energy consumption pattern, confirming that choosing the rule-based path meaningfully reduces the environmental impact of automated code optimization at scale [5].

10.3 LLM vs Rule-Based: Execution Latency

Figure 6 illustrates the execution latency comparison across test cases. The LLM mode records latencies from 2,400 ms to 7,200 ms, whereas the rule-based mode completes the same tasks in 45 ms to 450 ms — consistently an order of magnitude faster. The TC-05 Loop Invariant case shows the starkest gap (7,200 ms vs. 450 ms), underscoring that for latency-sensitive workflows the rule-based path is not only safer but also dramatically more responsive.

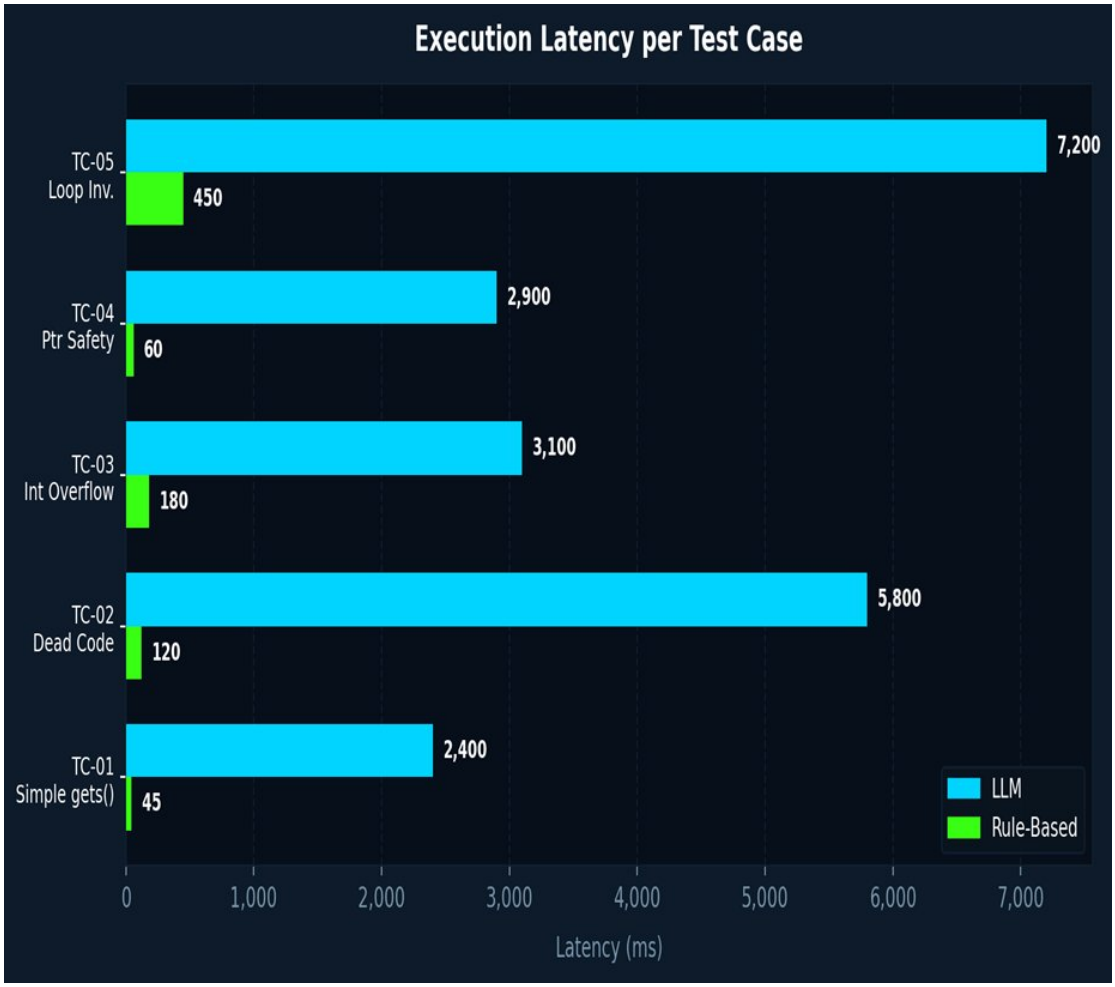


Figure 6: Execution latency (ms) per test case: LLM mode vs rule-based mode.

10.4 Original vs Optimized: Energy Usage

Figure 7 shows that DeepCompile's rule-based optimization reduces energy consumption across all test cases — from 20.7 mWh to 10.6 mWh for TC-01 (Buffer Overflow), and from 78.4 mWh to 24.5 mWh for TC-04 (Loop Hoisting). The average project run achieves a reduction from 77.1 mWh to 14.2 mWh, representing an approximate 82% energy saving. These reductions are a direct consequence of eliminating redundant operations, tightening loop bounds, and removing dead code during the optimization pass.

10.5 Original vs Optimized: Carbon Emissions

Figure 8 presents the corresponding carbon emission reductions. Original code emissions range from 8.7 mg CO_2e for simple buffer overflow patterns up to 32.9 mg for loop hoisting cases. After optimization, emissions drop to between 4.4 mg and 10.3 mg CO_2e across the same cases,

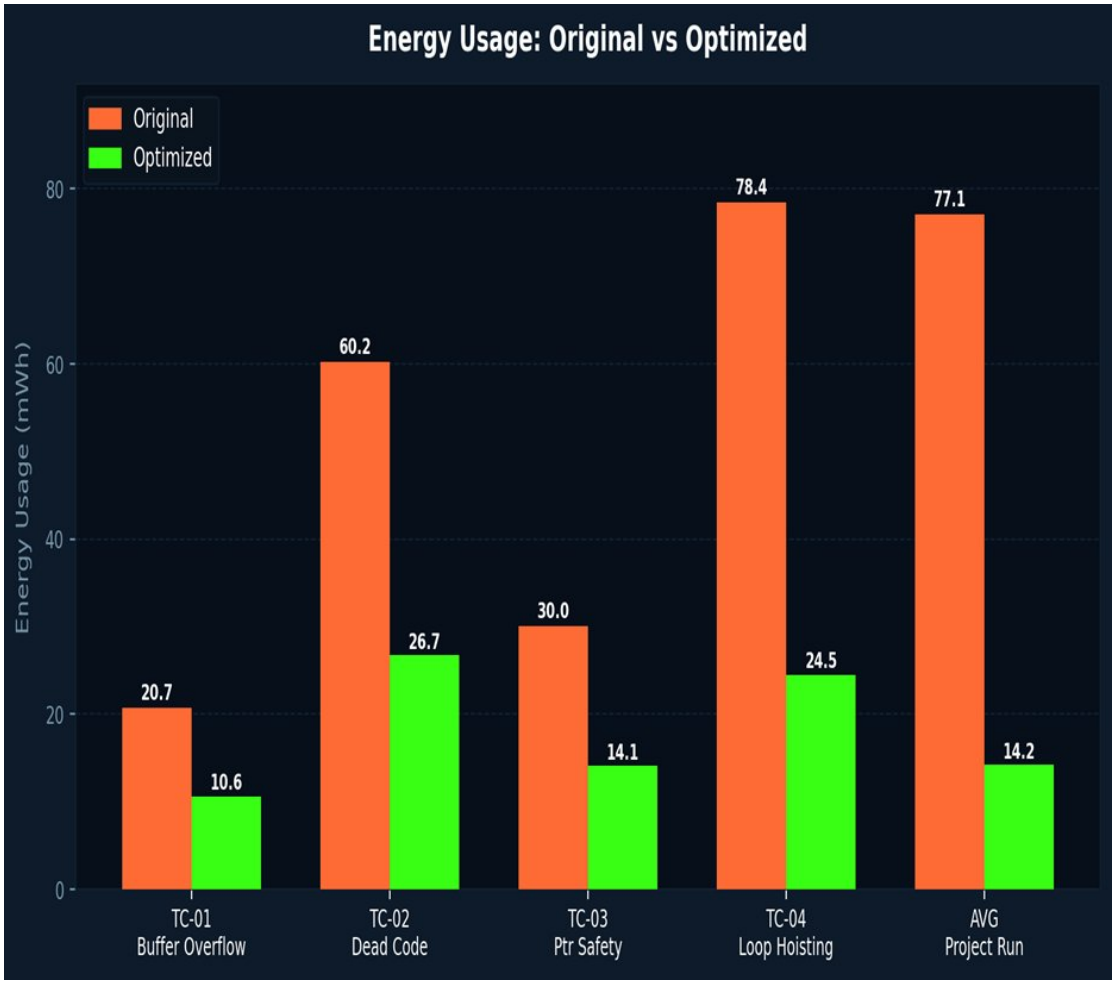


Figure 7: Energy usage (mWh): original unoptimized code vs optimized output across four test cases.

with the average project run falling from 32.4 mg to 5.9 mg — a reduction of over 81%. This confirms that source-to-source optimization, when governed by a safe and deterministic workflow, delivers measurable sustainability benefits beyond correctness and performance alone.

10.6 Overall Session Averages

Figure 9 summarizes the normalized session averages across all five test cases. The rule-based mode achieves average energy of 4.7 mWh versus 31.9 mWh for LLM, average CO_2 of 1.96 mg versus 13.41 mg, and average latency of 171 ms versus 4,280 ms. These results collectively confirm that the deterministic rule-based optimization path is the more sustainable and practical choice for production-oriented usage, while the LLM path remains available for cases that fall outside the rule coverage boundary.

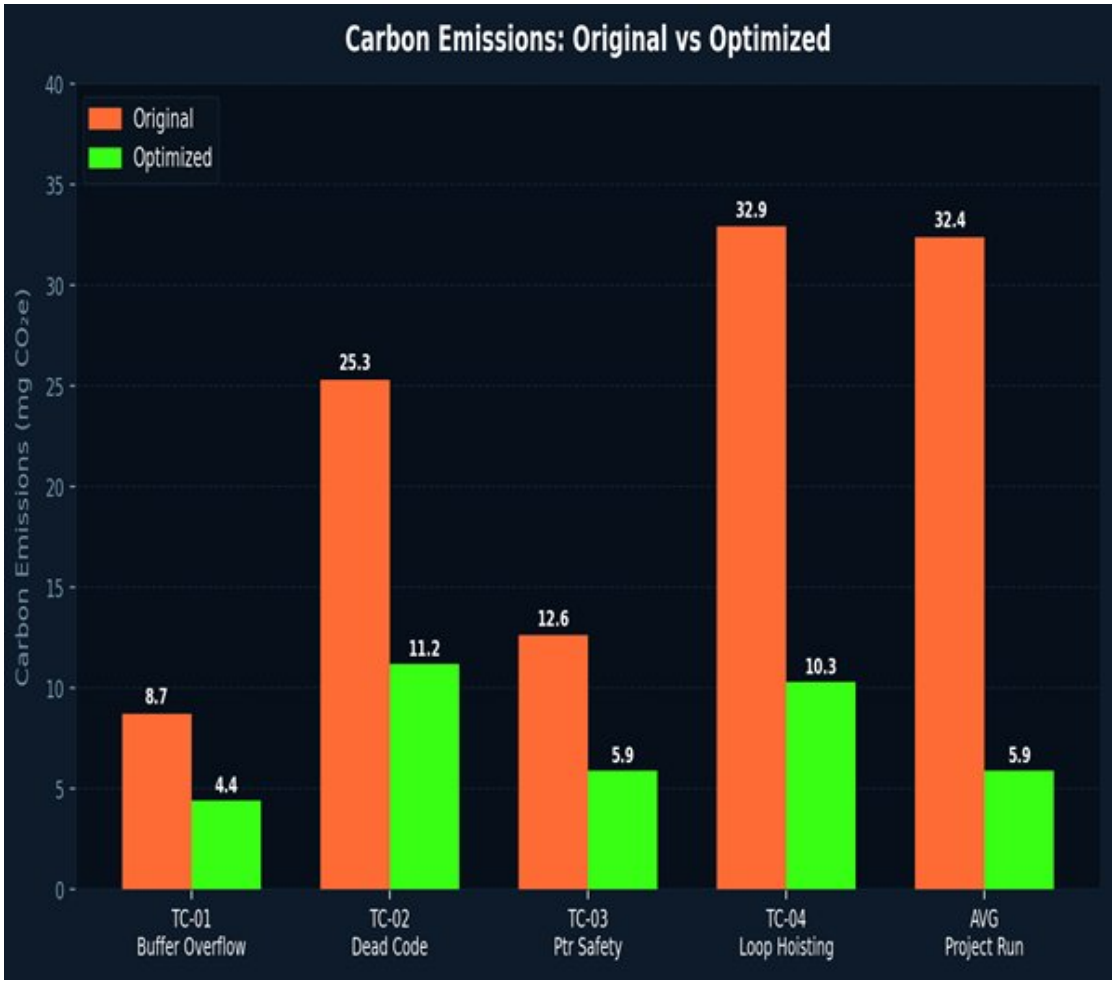


Figure 8: Carbon emissions (mg CO_2e): original unoptimized code vs optimized output across four test cases.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.

[2] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[3] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems*, 2008.

[4] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, 1976.

[5] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, 2020.

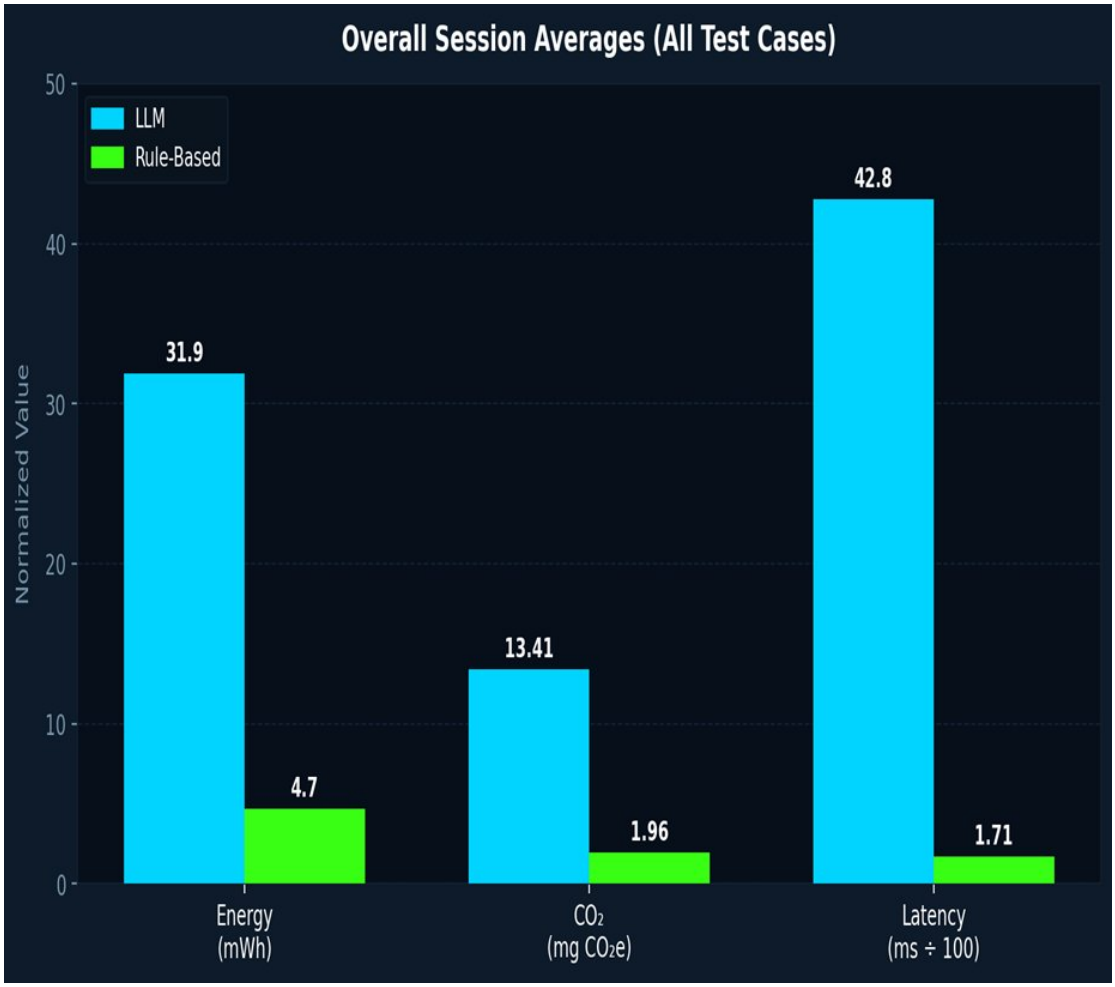


Figure 9: Normalized session averages across all test cases for energy, CO_2, and latency.

[6] C. Cummins et al., "CompilerGym: Robust, performant compiler optimization environments for machine learning," arXiv preprint arXiv:2109.08267, 2020.

[7] E. S. Babu, *PBL Report Writing Guidelines*, v1.1. Department of Computer Science and Engineering, NIT Warangal.

[8] Akshay Vishnu, Project artifacts in `Technical Deliverables and Documents/` and `evaluation/`, accessed locally from the DeepCompile repository, April 2026.

A Main Repository Paths

- `app.py`: Flask backend and SSE entry point
- `run_pipeline.py`: CLI entry point
- `src/agents/analysis_agent.py`
- `src/agents/optimization_agent.py`
- `src/agents/verification_agent.py`
- `src/agents/security_agent.py`
- `src/agents/refinement_agent.py`
- `src/agents/orchestrator_agent.py`
- `src/parser/code_parser.py`
- `src/reporting/report_contract.py`
- `frontend/src/App.jsx`

B Deterministic Commands Used for Validation

```
python -m unittest tests.test_orchestration_reporting -v
python -m unittest tests.test_sustainability_estimator -v
python evaluation/generate_week13_reports.py
python evaluation/verify_deliverables.py
npm run build
```

C Benchmark Cases Used in the Final Report Set

- `TC01_uninit_arithmetic.cpp`
- `TC07_div_by_zero_var.cpp`
- `TC11_buffer_overflow_loop.cpp`
- `TC12_strcpy_overflow.cpp`

- TC14_memory_leak.cpp
- TC16_use_after_free.cpp
- TC20_leak_in_loop.cpp
- TC27_int_overflow.cpp
- TC31_off_by_one.cpp
- TC35_unreachable_code.cpp
- TC39_empty_if_body.cpp
- TC41_leak_early_return.cpp

D Notes on Scope Honesty

The repository contains optional LLM paths and verification helpers that can use external tooling. The final report does not score the project on those optional paths. It instead follows the revised submission boundary already documented in the final dossier and evaluation artifacts: source-to-source optimization, deterministic rule-based evidence, and explicit workflow governance.